

Agenda.

1. Pre-requisites
2. Install Docker Compose on Centos
3. Create the example application
4. Test the app with Compose

Prerequisites

Ensure that you met the following prerequisites before continuing with this tutorial:

Docker - Compose

- Logged in as a **user with sudo privileges**.
- Have **Docker installed on your CentOS 7** system.

Install Docker Compose on CentOS

The recommended method for installing Docker Compose on CentOS 7 is by downloading the Compose binary from the Docker's GitHub repository.

At the time of writing this article, the latest stable version of Docker Compose is version 1.23.1. Before downloading the Compose binary visit the Compose repository release page on GitHub and check if there is a new version available for download.

Complete the following steps to install Docker Compose on CentOS 7:

1. Start by downloading the Docker Compose binary into the `/usr/local/bin` directory using the following curl command:

```
sudo curl -SL \
https://github.com/docker/compose/releases/latest/download/docker-
compose-linux-x86_64 \ -o /usr/local/lib/docker/cli-plugins/docker-
compose
```

2. Once the download is complete, make the binary executable by typing:

```
sudo chmod +x /usr/local/lib/docker/cli-plugins/docker-compose
```

3. To verify the installation type the following command to print the Compose version:

```
docker compose --version
```

The output will look something like this:

```
docker compose version 1.23.1, build b02f1306
```

Create the example application

The app used in this guide is based on the hit counter app in the [Get started with Docker Compose](#) guide. It consists of a Python app which maintains a counter in a Redis instance and increments the counter whenever you visit it.

Docker - Compose

1. Create a directory for the project:

```
$ mkdir stackdemo  
$ cd stackdemo
```

2. Create a file called `app.py` in the project directory and paste this in:

```
from flask import Flask  
from redis import Redis  
  
app = Flask(__name__)  
redis = Redis(host='redis', port=6379)  
  
@app.route('/')  
def hello():  
    count = redis.incr('hits')  
    return 'Hello World! I have been seen {} times.\n'.format(count)  
  
if __name__ == "__main__":  
    app.run(host="0.0.0.0", port=8000, debug=True)
```

3. Create a file called `requirements.txt` and paste these two lines in:

```
flask  
redis
```

4. Create a file called `Dockerfile` and paste this in:

```
FROM python:3.9-alpine  
ADD . /code  
WORKDIR /code  
RUN pip install -r requirements.txt  
CMD ["python", "app.py"]
```

Docker - Compose

5. Create a file called `docker-compose.yml` and paste this in:

```
version: '3.9'

services:
  web:
    image: vishwacloudlab/stackdemo
    build: .
    ports:
      - "8000:8000"
  redis:
    image: redis:alpine
```

The image for the web app is built using the Dockerfile defined above. It's also tagged with `127.0.0.1:5000` - the address of the registry created earlier. This is important when distributing the app to the swarm.

Test the app with Compose

1. Start the app with `docker-compose up`. This builds the web app image, pulls the Redis image if you don't already have it, and creates two containers.

You see a warning about the Engine being in swarm mode. This is because Compose doesn't take advantage of swarm mode, and deploys everything to a single node. You can safely ignore this.

Docker - Compose

```
$ docker compose up -d
```

WARNING: The Docker Engine you're using is running in swarm mode.

Compose does not use swarm mode to deploy services to multiple nodes in a swarm. All containers are scheduled on the current node.

To deploy your application across the swarm, use `docker stack deploy`.

Creating network "stackdemo_default" with the default driver

Building web

...(build output)...

Creating stackdemo_redis_1

Creating stackdemo_web_1

2. Check that the app is running with `docker-compose ps`:

3.

```
$ docker compose ps
```

4.

5.	Name	Command	State	Ports
----	------	---------	-------	-------

6.	-----			

7.	stackdemo_redis_1	docker-entrypoint.sh redis ...	Up	6379/tcp
----	-------------------	--------------------------------	----	----------

8.	stackdemo_web_1	python app.py	Up	0.0.0.0:8000->8000/tcp
----	-----------------	---------------	----	------------------------

You can test the app with `curl`:

```
$ curl http://localhost:8000
```

Hello World! I have been seen 1 times.

```
$ curl http://localhost:8000
```

Hello World! I have been seen 2 times.

```
$ curl http://localhost:8000
```

Docker - Compose

```
Hello World! I have been seen 3 times.
```

9. Bring the app down:

```
10. $ docker compose down
```

```
Stopping stackdemo_web_1 ... done
Stopping stackdemo_redis_1 ... done
Removing stackdemo_web_1 ... done
Removing stackdemo_redis_1 ... done
Removing network stackdemo_default
```

Scaling of the containers

Now that we have to scale the containers.

We would need to change the docker compose file such that we can map multiple “web” containers to the port on the host machine.

```
version: '3'
services:
```

Docker - Compose

```
web:
  image: vishwacloudlab/stackdemo
  build: .
  ports:
    - "8000-8010:8000"
redis:
  image: redis:alpine
```

```
[root@linux-client stackdemo]# docker compose up --scale web=3 -d
```

Output

Try the output with

http:// <linux-client>:8000

http:// <linux-client>:8001

http:// <linux-client>:8002

